

TJSC 2026 - Analista de Sistemas

Arquitetura de Sistemas e Integração

Aula 9 - Docker, containers, Kubernetes e Rancher

Banca FGV | Material autoral orientado ao edital vigente

Recorte desta aula

Cobre os itens 1.15 Containers Docker e 1.16 Orquestração com Kubernetes e Rancher da disciplina Arquitetura de Sistemas e Integração. O item 1.17, ambientes distribuídos e escaláveis, será aprofundado na Aula 10, mas aparece aqui apenas quando for necessário para entender containers e orquestração.

Objetivo de prova

Converter sua vivência com esteiras, APIs, Java/Quarkus, Angular e ambiente corporativo em acerto FGV: distinguir imagem, container, pod, deployment, service, cluster, registry, volume, namespace, Rancher e Kubernetes sem cair em alternativas quase certas.

1. Como a FGV tende a cobrar esta aula

A FGV costuma cobrar tecnologia em forma de decisão arquitetural: uma equipe precisa publicar uma API, escalar um serviço, trocar configuração sem reconstruir imagem, expor uma aplicação dentro ou fora do cluster, ou administrar vários clusters. A alternativa correta geralmente não é o termo mais moderno, mas o componente que cumpre exatamente aquela função.

Nesta aula, a banca pode explorar confusões de escopo: Docker empacota e executa containers; Kubernetes orquestra workloads containerizados em cluster; Rancher ajuda a gerenciar clusters Kubernetes. Esses três termos se relacionam, mas não são sinônimos.

Ponto de atenção FGV

Quando o enunciado falar em empacotar aplicação e dependências, pense em imagem/container. Quando falar em estado desejado, réplicas, rollout e self-healing em cluster, pense em Kubernetes. Quando falar em gestão centralizada de múltiplos clusters, políticas, usuários, visibilidade e UI, pense em Rancher.

2. Termos essenciais antes da arquitetura

Termo	Sentido para a prova	Armadilha comum
Container	Instância executável e isolada de uma imagem, com processo, filesystem próprio e configuração.	Achar que é uma máquina virtual completa.
Imagem	Template imutável, em camadas, com instruções e artefatos para criar containers.	Confundir imagem parada com container em execução.
Dockerfile	Arquivo declarativo com instruções para construir uma imagem.	Tratar como script de runtime do container.
Registry	Repositório para armazenar e distribuir imagens, público ou privado.	Confundir com repositório Git ou com volume persistente.
Volume	Mecanismo para persistir ou compartilhar dados além do ciclo de vida do container.	Achar que todo dado dentro do container persiste após sua remoção.
Pod	Menor unidade implantável do Kubernetes; pode conter um ou mais containers co-localizados.	Achar que pod e container são exatamente a mesma coisa.
Deployment	Objeto que declara estado desejado para Pods e ReplicaSets, com rollout, rollback e escala.	Achar que é o objeto que expõe rede externa.
Service	Abstração estável de rede para acessar um conjunto de Pods por selector/endpoints.	Confundir com Deployment ou API Gateway.
Cluster	Conjunto de nós controlado pelo Kubernetes, com plano de controle e nós de trabalho.	Achar que cluster é apenas uma aplicação.
Rancher	Plataforma de gestão de Kubernetes, útil para administrar clusters, usuários, políticas, visibilidade e catálogos.	Achar que Rancher substitui o Kubernetes como orquestrador.

3. Docker e containers

Docker é uma plataforma para desenvolver, empacotar, distribuir e executar aplicações em containers. O valor arquitetural do container está em tornar a unidade de execução mais previsível: a aplicação, suas bibliotecas, runtime e configuração de inicialização passam a ser transportados como artefato padronizado.

Para prova, container não é só “um processo”. Ele é uma forma de isolamento em nível de sistema operacional. Em geral, compartilha o kernel do host, diferentemente de uma máquina virtual tradicional, que virtualiza um sistema operacional convidado inteiro. Isso explica por que containers tendem a ser mais leves e rápidos, mas também por que exigem cuidado com isolamento, permissões, imagens vulneráveis e segredos.

Comparação	Container	Máquina virtual
Unidade principal	Processo isolado com filesystem, rede e dependências empacotadas.	Sistema operacional convidado completo sobre hipervisor.
Kernel	Normalmente compartilha o kernel do host.	Cada VM possui seu próprio sistema operacional convidado.
Peso/tempo de inicialização	Mais leve e rápido em geral.	Mais pesado em geral.
Portabilidade	Alta, desde que runtime e arquitetura sejam compatíveis.	Boa, mas artefatos tendem a ser maiores.
Segurança	Isolamento útil, mas não deve ser tratado como barreira absoluta.	Isolamento mais forte em muitos cenários.
Uso típico em prova	Entrega de APIs, microsserviços, jobs, workers e aplicações padronizadas.	Ambientes com necessidade de isolamento forte ou sistemas legados completos.

3.1 Imagem, container, camada e registry

Imagem é um artefato somente leitura usado como molde. Container é a instância em execução ou criada a partir dessa imagem. Registry é o local onde imagens são armazenadas e distribuídas. Tag é um rótulo de versão, como minha-api:1.4.2. Camadas reduzem trabalho de rebuild e transferência quando instruções anteriores do Dockerfile não mudam.

Exemplo corporativo

Uma API Quarkus de consulta processual pode ser empacotada em uma imagem tjsc/processos-api:2.1.0. Em homologação e produção, o cluster executa containers a partir dessa imagem. Se a configuração de URL do banco variar por ambiente, ela deve entrar por variável, Secret ou ConfigMap, não ficar fixa dentro da imagem.

```
Dockerfile simplificado - foco conceitual
FROM eclipse-temurin:17-jre
WORKDIR /app
COPY target/processos-runner.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

O Dockerfile acima não é o ponto central da prova; o ponto é saber que ele descreve a construção da imagem. O container será criado depois, em runtime, a partir dessa imagem. Em um cluster Kubernetes, a imagem normalmente será referenciada dentro de um manifesto de Pod, Deployment, Job ou outro workload.

3.2 Container é efêmero: estado deve ser tratado explicitamente

A FGV pode derrubar candidato com a ideia de persistência. Alterações feitas no filesystem interno do container podem desaparecer quando ele é removido, recriado ou substituído. Dados importantes devem ser enviados a banco, object storage, volume ou mecanismo persistente apropriado.

Isso é especialmente relevante para aplicações Java/Quarkus e Angular: o front-end Angular empacotado em Nginx pode ser stateless; a API Quarkus também deve ser tratada como stateless sempre que possível; já banco de dados, arquivos e filas exigem persistência e cuidado operacional.

4. Kubernetes: orquestração de containers

Kubernetes, também chamado K8s, é uma plataforma de orquestração para automatizar implantação, escalabilidade e gerenciamento de aplicações containerizadas. Em vez de executar containers manualmente em um único host, o Kubernetes trabalha com estado desejado: você declara quantas réplicas quer, qual imagem usar, quais portas expor e quais restrições aplicar; os controladores tentam reconciliar o estado real ao estado desejado.

O Kubernetes não é apenas “rodar Docker em vários servidores”. Ele adiciona um plano de controle, uma API declarativa, objetos próprios, mecanismos de escalabilidade, health checks, rede de serviços, atualização progressiva, rollback e integração com armazenamento e observabilidade.

4.1 Cluster, control plane e worker nodes

Componente	Função conceitual	Como cai em prova
Cluster	Conjunto de nós gerenciados como uma unidade.	Ambiente onde workloads containerizados são executados e orquestrados.
Control plane	Toma decisões globais: agenda workloads, mantém estado desejado e expõe API.	Não roda "a aplicação" necessariamente; coordena o cluster.
API Server	Porta de entrada da API do Kubernetes.	kubectl, controladores e ferramentas conversam com ele.
etcd	Armazena o estado do cluster de forma consistente.	Não é banco relacional da aplicação.
Scheduler	Escolhe em qual nó um Pod deve rodar.	Não balanceia tráfego HTTP do usuário final.
Controller Manager	Executa controladores que reconciliam estado desejado e real.	Está associado a automação e self-healing.
Worker node	Nó onde Pods normalmente executam.	Possui kubelet, runtime de container e componentes de rede.
kubelet	Agente do nó que garante execução dos containers descritos nos Pods.	Não substitui API Gateway nem Service.

4.2 Pod: a menor unidade implantável

Pod é a menor unidade implantável do Kubernetes. Um Pod agrupa um ou mais containers que compartilham rede e volumes e são agendados juntos. A maioria dos sistemas usa um container principal por Pod, mas sidecars podem aparecer para log, proxy, segurança ou coleta de métricas.

Pegadinha de prova

Se a questão perguntar a menor unidade implantável do Kubernetes, responda Pod, não container. O container é a unidade de runtime; o Pod é o objeto mínimo que o Kubernetes cria e gerencia como workload.

4.3 Deployment, ReplicaSet e rollout

Deployment é o objeto mais comum para aplicações stateless. Ele descreve o estado desejado dos Pods, como imagem, réplicas, labels e estratégia de atualização. O ReplicaSet mantém o número de réplicas. O Deployment cria e controla ReplicaSets para permitir rollout e rollback.

Exemplo textual de Deployment

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: processos-api
spec:
  replicas: 3
  selector:
    matchLabels:
      app: processos-api
  template:
    metadata:
      labels:
        app: processos-api
    spec:
      containers:
        - name: api
          image: registry.interno/processos-api:2.1.0
          ports:
            - containerPort: 8080
```

A leitura FGV desse YAML é: kind: Deployment indica objeto de workload; replicas: 3 indica três Pods desejados; selector/matchLabels vincula o Deployment aos Pods; image indica a imagem de container; containerPort informa a porta dentro do container. Isso não expõe automaticamente a aplicação para outros clientes: para isso, normalmente entra um Service e, se for acesso HTTP externo, Ingress ou Gateway/API Gateway.

4.4 Service, DNS e exposição de aplicações

Pods são efêmeros: podem morrer, ser recriados e mudar de IP. O Service cria uma abstração estável para um conjunto de Pods, normalmente selecionados por labels. Ele permite que outros componentes acessem "processos-api" sem conhecer cada IP de Pod.

Objeto	Função	Não confundir com
Service ClusterIP	Expõe serviço dentro do cluster por IP/DNS	Não expõe necessariamente para Internet.

	interno.	
NodePort	Abre uma porta em cada nó para acesso externo simples.	Não é a opção mais refinada para roteamento HTTP.
LoadBalancer	Pede um balanceador externo, comum em provedores cloud.	Não é um Deployment.
Ingress	Regras HTTP/HTTPS de entrada para múltiplos serviços.	Ingress não é Service; depende de um Ingress Controller.
DNS interno	Permite resolver nomes de Services no cluster.	Não substitui labels/selectors.

4.5 ConfigMap, Secret, volumes e persistência

ConfigMap guarda configuração não sensível, como URLs, flags e parâmetros. Secret guarda dados sensíveis, como senhas e tokens, mas isso não autoriza tratar Secret como “cofre mágico”: em ambientes reais, é preciso cuidar de RBAC, criptografia em repouso, rotação e integração com vaults quando exigido.

Volumes, PersistentVolume (PV), PersistentVolumeClaim (PVC) e StorageClass aparecem quando a aplicação precisa persistir dados. Para APIs stateless, a tendência é manter estado fora do Pod. Para bancos, filas e storage, o tema exige cuidado maior e geralmente envolve StatefulSet ou serviços gerenciados.

Necessidade	Objeto/abordagem provável	Pegadinha
Trocar URL de API externa sem rebuild da imagem.	ConfigMap ou variável de ambiente.	Não reconstruir imagem para cada ambiente.
Senha de banco.	Secret, idealmente com políticas e criptografia em repouso.	Secret não é só “segurança automática”.
Guardar arquivos gerados por serviço.	Volume/PVC ou storage externo.	Não confiar no filesystem efêmero do container.
Manter 3 instâncias de API sem estado.	Deployment com replicas: 3.	Não usar StatefulSet sem necessidade.
Banco com identidade estável e armazenamento por réplica.	StatefulSet/PVC ou serviço gerenciado.	Deployment nem sempre é adequado para estado.

4.6 Health checks: liveness, readiness e startup probes

Probes ajudam o Kubernetes a decidir se o container está vivo, pronto para receber tráfego ou ainda inicializando. A FGV pode cobrar a diferença: liveness probe indica se o container deve ser reiniciado; readiness probe indica se o Pod deve receber tráfego; startup probe protege aplicações lentas na inicialização, evitando que a liveness mate prematuramente o processo.

Probe	Pergunta respondida	Consequência típica
Liveness	O processo está travado ou morto?	Se falhar, o container pode ser reiniciado.
Readiness	A aplicação está pronta para receber tráfego?	Se falhar, o Pod sai dos endpoints do Service.
Startup	A aplicação ainda está inicializando?	Enquanto não passar, evita falhas prematuras de liveness.

5. Rancher: gestão de clusters Kubernetes

Rancher é uma plataforma de gestão de containers/Kubernetes para organizações que operam workloads em produção. O foco de prova é entendê-lo como camada de gerenciamento: criação/importação de clusters, autenticação centralizada, controle de acesso, políticas, projetos, namespaces, visibilidade, monitoramento e catálogos.

Rancher não transforma automaticamente uma aplicação em container, não substitui o Kubernetes como orquestrador e não é sinônimo de Docker. **Ele facilita a administração de ambientes Kubernetes**, inclusive múltiplos clusters em nuvem pública, privada, híbrida ou edge.

Pergunta de prova	Resposta mais aderente
Quem empacota uma aplicação em imagem e executa container localmente?	Docker ou runtime compatível.
Quem orquestra Pods, Deployments e Services em cluster?	Kubernetes.
Quem centraliza gestão, RBAC, visibilidade e múltiplos clusters?	Rancher.
Quem mantém réplicas e faz rollout de Pods stateless?	Deployment no Kubernetes.
Quem fornece endpoint estável para um conjunto de Pods?	Service no Kubernetes.

6. Exemplo integrado: Angular + Quarkus + Kubernetes

Imagine um sistema do TJSC com front-end Angular e API Java/Quarkus para consulta de processos. Em uma arquitetura containerizada, o Angular pode ser compilado e servido por Nginx em um container; a API Quarkus pode rodar em outro container;

cada um pode ter sua própria imagem e seu próprio Deployment. O Kubernetes não substitui a API nem o Angular; ele coordena execução, escala, rede e atualização desses workloads.

Camada do sistema	Artefato provável	Objeto Kubernetes provável
Front-end Angular estático	Imagem com Nginx servindo arquivos buildados.	Deployment + Service + Ingress.
API Quarkus	Imagem com aplicação Java e runtime.	Deployment + Service + probes.
Configuração por ambiente	Variáveis, propriedades e URLs.	ConfigMap e Secret.
Logs	stdout/stderr coletados por stack de observabilidade.	Integração com logging do cluster.
Banco de dados	Serviço externo ou stateful gerenciado.	Não necessariamente dentro do cluster.

Leitura de diagrama textual

Usuário -> Ingress -> Service frontend -> Pods Angular/Nginx -> Service processos-api -> Pods Quarkus -> banco externo. O Service não executa aplicação; ele estabiliza acesso. O Deployment não roteia HTTP externo; ele mantém réplicas. O Ingress não substitui o Service; ele normalmente direciona tráfego HTTP/HTTPS para Services.

7. Trade-offs que derrubam candidato

Decisão	Benefício	Custo/risco
Containerizar aplicação	Padroniza ambiente e facilita CI/CD.	Exige gestão de imagens, segurança, registry e configuração.
Usar Kubernetes	Automação, escala, rollout, self-healing e abstrações de rede.	Aumenta complexidade operacional, rede, observabilidade e governança.
Aumentar réplicas	Melhora capacidade e disponibilidade de aplicação stateless.	Não resolve gargalo de banco, estado compartilhado ou bug lógico.
Mover estado para dentro do cluster	Controle unificado em alguns cenários.	Persistência, backup, disaster recovery e performance ficam mais críticos.
Usar Rancher	Gestão centralizada e visibilidade de múltiplos clusters.	Adiciona camada operacional e dependência de plataforma de gestão.
Usar ConfigMap/Secret	Separa configuração da imagem.	Exige governança de acesso e rotação de segredo.

8. Pegadinhas FGV

- Imagem não é container: imagem é molde; container é instância executável.
- Container não é máquina virtual completa: normalmente compartilha kernel do host.
- Pod não é exatamente sinônimo de container: Pod pode ter um ou mais containers e é a menor unidade gerenciada pelo Kubernetes.
- Deployment não expõe rede por si só: ele gerencia Pods/ReplicaSets; Service/Ingress/Gateway tratam exposição e roteamento.
- Service não cria réplicas: ele cria endpoint estável para Pods selecionados.
- Ingress não é Service: é recurso de roteamento HTTP/HTTPS que depende de controller.
- Kubernetes não é Docker: Docker é plataforma/runtime/ecossistema de containers; Kubernetes é orquestrador.
- Rancher não substitui Kubernetes: gerencia clusters Kubernetes e recursos associados.
- Escala horizontal não corrige aplicação stateful mal desenhada nem gargalo de banco.
- Secret não deve ser tratado como proteção absoluta: depende de RBAC, criptografia em repouso e práticas de segurança.
- Volume resolve persistência, mas não resolve backup, consistência ou alta disponibilidade sozinho.
- Liveness probe reinicia processo; readiness probe controla recebimento de tráfego.

9. Questões autorais - padrão FGV

Resolva as 20 questões sem consultar o gabarito. As questões são autorais e calibradas pelo estilo de cobrança da FGV: cenário técnico, alternativas próximas e foco em distinção fina de papel arquitetural.

Questão 1. Em uma equipe de desenvolvimento, foi criado um arquivo com instruções para montar uma imagem contendo uma aplicação Java, suas dependências e o comando de inicialização. Esse arquivo será usado antes da execução do container. O elemento descrito é, mais adequadamente, o:

- Dockerfile.
- Pod.
- Service.

- D) Namespace.
- E) Ingress.

Questão 2. Considere uma API containerizada em execução. Durante investigação de incidente, um analista afirma que “a imagem está respondendo requisições na porta 8080”. Tecnicamente, a formulação é imprecisa porque:

- A) imagens respondem apenas a requisições TCP, enquanto containers respondem a HTTP.
- B) a imagem é um template; quem executa e responde em runtime é o container criado a partir dela.
- C) imagens só existem no Kubernetes, e containers só existem no Docker.
- D) a imagem substitui o Dockerfile após o primeiro build.
- E) containers não podem responder requisições; apenas Services respondem.

Questão 3. Em Kubernetes, deseja-se manter três instâncias de uma API Quarkus sem estado, substituir gradualmente uma versão por outra e possibilitar rollback caso a atualização falhe. O objeto mais diretamente relacionado a esse gerenciamento é:

- A) ConfigMap.
- B) Secret.
- C) Deployment.
- D) PersistentVolume.
- E) Namespace.

Questão 4. Em um cluster Kubernetes, vários Pods que executam a mesma API podem ser destruídos e recriados, mudando seus endereços IP. Para fornecer um endpoint estável a outros componentes do cluster, a solução típica é utilizar:

- A) Service.
- B) Dockerfile.
- C) ReplicaSet diretamente exposto ao usuário.
- D) Volume.
- E) Image tag.

Questão 5. Sobre Pods no Kubernetes, assinale a afirmativa correta.

- A) Pod é sempre equivalente a uma máquina virtual completa com sistema operacional próprio.
- B) Pod é a menor unidade implantável gerenciada pelo Kubernetes e pode conter um ou mais containers.
- C) Pod é o repositório remoto usado para armazenar imagens Docker.
- D) Pod é usado exclusivamente para tráfego HTTP externo.
- E) Pod substitui o Deployment em qualquer cenário de escala horizontal.

Questão 6. Uma aplicação Angular estática e uma API Java foram empacotadas em imagens distintas e publicadas em registry privado. Em seguida, cada imagem passou a ser usada por workloads no Kubernetes. Nessa situação, é correto afirmar que:

- A) registry é o mecanismo que mantém três réplicas em execução.
- B) imagem e container são termos equivalentes quando usados em ambiente Kubernetes.
- C) o Kubernetes dispensa qualquer mecanismo de rede entre os Pods.
- D) as imagens funcionam como artefatos de distribuição, enquanto os containers são instâncias em execução criadas a partir delas.
- E) o Dockerfile é o objeto responsável por balancear requisições entre Pods.

Questão 7. Analise as afirmativas sobre containers e Kubernetes. I. Containers tendem a compartilhar o kernel do host, diferentemente de máquinas virtuais tradicionais. II. Kubernetes trabalha com a ideia de estado desejado e reconciliação entre estado declarado e estado observado. III. Um Service Kubernetes é o objeto responsável por construir a imagem da aplicação a partir de um Dockerfile. Está correto o que se afirma em:

- A) I, apenas.

- B) II, apenas.
- C) I e II, apenas.
- D) II e III, apenas.
- E) I, II e III.

Questão 8. Em um manifesto Kubernetes, o campo kind apresenta o valor Service, e o spec contém selector e ports. A interpretação mais adequada é que esse recurso:

- A) define a construção da imagem a partir de camadas.
- B) declara a quantidade desejada de réplicas de Pods.
- C) agrupa e expõe logicamente um conjunto de endpoints, frequentemente Pods selecionados por labels.
- D) armazena senha de banco em formato criptográfico obrigatório.
- E) define a máquina física na qual o container será sempre executado.

Questão 9. Uma equipe quer separar a URL de um serviço externo da imagem do container, para que a mesma imagem seja usada em homologação e produção com configurações diferentes. No Kubernetes, uma abordagem típica para configuração não sensível é usar:

- A) PersistentVolumeClaim.
- B) ConfigMap.
- C) Ingress Controller.
- D) ReplicaSet.
- E) Docker tag latest obrigatoriamente.

Questão 10. No contexto de Kubernetes, uma liveness probe é mais bem descrita como mecanismo para:

- A) impedir que qualquer container seja reiniciado automaticamente.
- B) verificar se a aplicação está viva, permitindo reinício do container quando a verificação falha.
- C) selecionar o nó mais barato para executar o Pod.
- D) armazenar logs de auditoria em banco relacional.
- E) expor tráfego HTTP externo com TLS.

Questão 11. Uma readiness probe falha temporariamente em um Pod de uma API. Considerando o comportamento esperado, a consequência mais aderente é:

- A) o Pod deve ser removido dos endpoints que recebem tráfego até voltar a estar pronto.
- B) a imagem deve ser reconstruída automaticamente.
- C) o cluster deve excluir o namespace completo.
- D) o Service deve virar um Deployment.
- E) o Dockerfile deve ser alterado pelo kubelet.

Questão 12. Relacione os conceitos às descrições: 1. Registry. 2. Deployment. 3. Rancher. 4. Service. () Armazena e distribui imagens. () Gerencia declarativamente Pods e ReplicaSets. () Atua como plataforma de gestão de clusters Kubernetes. () Fornece endpoint lógico para conjunto de Pods. A sequência correta é:

- A) 1 - 2 - 3 - 4.
- B) 2 - 1 - 4 - 3.
- C) 4 - 3 - 2 - 1.
- D) 1 - 3 - 2 - 4.
- E) 3 - 2 - 1 - 4.

Questão 13. Um órgão público usa vários clusters Kubernetes, alguns em nuvem pública e outros em datacenter próprio. Precisa de autenticação centralizada, controle de acesso, gestão de projetos/namespaces e visibilidade operacional. O produto/abordagem mais diretamente ligado a essa necessidade, dentro do escopo da aula, é:

- A) Dockerfile multi-stage.
- B) Rancher.
- C) Container image layer.
- D) Liveness probe.
- E) NodePort isolado.

Questão 14. Assinale a alternativa INCORRETA sobre Docker, Kubernetes e Rancher.

- A) Docker pode ser usado para criar e executar containers.
- B) Kubernetes orquestra workloads containerizados em cluster.
- C) Rancher pode ajudar a gerir múltiplos clusters Kubernetes.
- D) Um Deployment é usualmente empregado para gerenciar réplicas de aplicações stateless.
- E) Rancher substitui o Kubernetes como mecanismo de orquestração de Pods, dispensando o cluster Kubernetes.

Questão 15. Em uma aplicação que grava arquivos temporários dentro do filesystem do container, a equipe percebe perda de arquivos após recriação dos Pods. A causa provável e a correção arquitetural mais adequada são:

- A) Service apaga os arquivos; usar selector fixo.
- B) o filesystem do container deve ser tratado como efêmero; usar volume, storage externo ou persistência apropriada.
- C) Ingress remove arquivos locais; usar NodePort.
- D) ConfigMap criptografa arquivos temporários; usar Secret.
- E) Rancher impede persistência por padrão; remover o cluster.

Questão 16. Observe o diagrama textual: Cliente externo -> Ingress -> Service processos-api -> Pods processos-api. A leitura correta é:

- A) Ingress constrói a imagem da API e envia ao registry.
- B) Service mantém as réplicas e faz rollout da versão.
- C) Pods são endpoints lógicos permanentes e nunca mudam.
- D) Ingress trata entrada HTTP/HTTPS, Service estabiliza acesso aos Pods e Pods executam os containers da aplicação.
- E) Deployment é dispensável sempre que existe Ingress.

Questão 17. Em relação ao escalonamento horizontal de uma API stateless em Kubernetes, assinale a alternativa mais adequada.

- A) Aumentar réplicas pode aumentar capacidade de atendimento da aplicação, mas não elimina gargalos externos como banco ou serviço legado.
- B) Aumentar réplicas garante consistência transacional em qualquer banco.
- C) Aumentar réplicas substitui autenticação, logs e observabilidade.
- D) Aumentar réplicas obriga cada Pod a ter IP público fixo.
- E) Aumentar réplicas transforma automaticamente a aplicação em microsserviços.

Questão 18. Uma equipe define Secret para armazenar credenciais de banco e acredita que isso, sozinho, resolve todos os riscos de sigilo. A avaliação técnica mais adequada é:

- A) Correta, porque Secret sempre criptografa dados em repouso e em uso sem configuração adicional.
- B) Correta, porque Secret torna RBAC desnecessário.
- C) Incompleta, pois Secret ajuda a separar dados sensíveis, mas deve ser combinado com controle de acesso, rotação, políticas e, quando necessário, criptografia em repouso/vault.
- D) Incorreta, porque Kubernetes não possui nenhum recurso para dados sensíveis.
- E) Incorreta, porque credenciais devem ficar sempre hardcoded na imagem para evitar vazamento por variável de ambiente.

Questão 19. Em um cluster Kubernetes, o scheduler tem como responsabilidade principal:

- A) receber requisições HTTP dos usuários externos.
- B) armazenar imagens no registry.
- C) decidir em qual nó um Pod será executado, considerando restrições e recursos.
- D) compilar o código Java da aplicação.
- E) substituir o banco de dados da aplicação.

Questão 20. Uma questão descreve uma solução em que: a aplicação é empacotada em imagem; três Pods devem ser mantidos em execução; um endpoint estável interno deve ser oferecido aos consumidores; e a equipe quer administrar clusters por interface com RBAC centralizado. A associação correta é:

- A) Imagem - Service; réplicas - Registry; endpoint - Dockerfile; gestão - Pod.
- B) Imagem - Docker/registry; réplicas - Deployment; endpoint - Service; gestão centralizada - Rancher.
- C) Imagem - Rancher; réplicas - Ingress; endpoint - ConfigMap; gestão - Dockerfile.
- D) Imagem - Namespace; réplicas - Secret; endpoint - Volume; gestão - container.
- E) Imagem - Service; réplicas - Service; endpoint - Deployment; gestão - Registry.

10. Gabarito resumido

1-A | 2-B | 3-C | 4-A | 5-B | 6-D | 7-C | 8-C | 9-B | 10-B | 11-A | 12-A | 13-B | 14-E | 15-B | 16-D | 17-A | 18-C | 19-C | 20-B

11. Comentários completos

Questão 1 - Gabarito: A. A correta é A: Dockerfile é o arquivo com instruções de build da imagem. B pode enganar porque Pod usa imagem, mas é objeto de runtime no Kubernetes. C é rede lógica, não build. D separa escopo/recursos. E trata entrada HTTP/HTTPS, não construção de imagem.

Questão 2 - Gabarito: B. A correta é B: imagem é template; container é a instância executável que responde em runtime. A cria distinção falsa entre TCP e HTTP. C é errada porque imagens e containers existem além do Kubernetes. D confunde Dockerfile e imagem. E é sedutora porque Service participa do acesso, mas quem executa a aplicação é o container no Pod.

Questão 3 - Gabarito: C. A correta é C: Deployment gerencia Pods/ReplicaSets, rollout, rollback e escala de aplicações stateless. A e B tratam configuração. D trata armazenamento. E organiza escopo lógico, não mantém réplicas.

Questão 4 - Gabarito: A. A correta é A: Service fornece endpoint estável para Pods efêmeros. B constrói imagem. C parece técnica, mas ReplicaSet não é o objeto adequado de exposição ao consumidor. D trata dados. E identifica versão de imagem, não rede.

Questão 5 - Gabarito: B. A correta é B: Pod é a menor unidade implantável do Kubernetes e pode conter um ou mais containers. A confunde com máquina virtual. C descreve registry. D confunde Pod com Ingress/Service. E erra porque Deployment continua importante para escala e ciclo de vida.

Questão 6 - Gabarito: D. A correta é D: imagens são artefatos distribuíveis; containers são instâncias em execução. A atribui ao registry função de orquestração. B apaga diferença essencial. C ignora Services, DNS e rede. E confunde Dockerfile com balanceamento.

Questão 7 - Gabarito: C. I está correta: containers geralmente compartilham kernel do host. II está correta: Kubernetes trabalha com estado desejado e reconciliação. III está errada: Service não constrói imagem; Dockerfile/build fazem isso. Logo, C.

Questão 8 - Gabarito: C. A correta é C: Service abstrai acesso a endpoints, geralmente Pods selecionados por labels. A é Dockerfile/build. B é Deployment/ReplicaSet. D é Secret, e ainda assim não "criptografia obrigatória" em todo cenário. E é afinidade/scheduler, não Service.

Questão 9 - Gabarito: B. A correta é B: ConfigMap é típico para configuração não sensível. A trata solicitação de armazenamento. C trata entrada HTTP. D mantém réplicas. E é má prática como solução de configuração e não separa parâmetros por ambiente.

Questão 10 - Gabarito: B. A correta é B: liveness verifica se o container está vivo e pode levar a reinício. A contraria a função. C é scheduler. D é logging/auditoria. E é Ingress/Gateway/TLS.

Questão 11 - Gabarito: A. A correta é A: readiness controla se o Pod está pronto para receber tráfego, retirando-o de endpoints quando falha. B confunde health check com build. C exagera efeito. D troca objetos. E atribui ao kubelet edição de Dockerfile.

Questão 12 - Gabarito: A. A sequência correta é 1-2-3-4. Registry armazena imagens; Deployment gerencia Pods/ReplicaSets; Rancher gerencia clusters; Service fornece endpoint lógico. As demais sequências trocam build, runtime, gestão e rede.

Questão 13 - Gabarito: B. A correta é B: Rancher atende gestão centralizada de clusters, acesso, projetos, namespaces e visibilidade. A é técnica de build. C é estrutura de imagem. D é health check. E expõe porta de nó, mas não resolve gestão multicuster/RBAC.

Questão 14 - Gabarito: E. A incorreta é E: Rancher não substitui Kubernetes; ele gerencia clusters Kubernetes. A, B, C e D são descrições adequadas dos papéis de Docker, Kubernetes, Rancher e Deployment.

Questão 15 - Gabarito: B. A correta é B: filesystem interno do container/Pod deve ser tratado como efêmero; persistência exige volume, storage externo ou solução apropriada. A, C e D atribuem perda a objetos errados. E inventa limitação absoluta do Rancher.

Questão 16 - Gabarito: D. A correta é D: Ingress recebe/roteia entrada HTTP/HTTPS; Service estabiliza acesso a Pods; Pods executam containers. A é build/registry. B é função de Deployment. C ignora efemeridade dos Pods. E é falso: Ingress não elimina necessidade de Deployment.

Questão 17 - Gabarito: A. A correta é A: escalar réplicas ajuda capacidade de aplicação stateless, mas não resolve automaticamente banco, legado, estado ou bugs. B, C, D e E transformam escala horizontal em solução universal, exatamente o tipo de exagero que a FGV explora.

Questão 18 - Gabarito: C. A correta é C: Secret separa dados sensíveis, mas precisa de RBAC, políticas, rotação e, conforme o ambiente, criptografia/vault. A exagera proteção automática. B torna RBAC desnecessário indevidamente. D ignora o recurso Secret. E sugere hardcode em imagem, prática inadequada.

Questão 19 - Gabarito: C. A correta é C: scheduler escolhe o nó para executar o Pod. A é Ingress/Service/API Gateway. B é registry. D é etapa de build/CI. E é banco de dados, fora do papel do scheduler.

Questão 20 - Gabarito: B. A correta é B: Docker/registry lidam com imagem; Deployment mantém réplicas; Service cria endpoint estável; Rancher gerencia clusters com RBAC centralizado. As demais alternativas embaralham objetos próximos, mas com papéis trocados.

12. Referências consultadas

- TJSC - Edital n. 10/2026 retificado: disciplina Arquitetura de Sistemas e Integração, itens 1.15 e 1.16.
- Manual Mestre de Calibração FGV para Aulas e Questões - Projeto TJSC 2026.
- Docker Docs - What is Docker? <https://docs.docker.com/get-started/docker-overview/>
- Kubernetes Documentation - Overview. <https://kubernetes.io/docs/concepts/overview/>
- Kubernetes Documentation - Pods. <https://kubernetes.io/docs/concepts/workloads/pods/>
- Kubernetes Documentation - Deployments. <https://kubernetes.io/docs/concepts/workloads/controllers/deployment/>
- Kubernetes Documentation - Service. <https://kubernetes.io/docs/concepts/services-networking/service/>
- Rancher Manager Documentation - Overview. <https://ranchermanager.docs.rancher.com/getting-started/overview>

Material autoral. As questões são inéditas e não reproduzem questões reais; foram construídas para treinar o estilo de cobrança, vocabulário e distinções conceituais da FGV.